

数据库测试实验报告（第16周）

班 级：3班

姓 名：梁力航

学 号：23336128

三、实验结果

(1) 第1题：创建表CARD

SQL语句：

```
CREATE TABLE CARD(  
    cid char(10) PRIMARY KEY,  
    sid char(10) REFERENCES students(sid),  
    money decimal(10,2)  
);
```

结果截图：

13

ER1] 1/6

1/13

ER2] 1/6

2

3

4 ✓

5

6

7

8

9

--1.创建表CARD(cid,sid,money),其中cid为主键,具有唯一性约束, sid为外键, 参照STUDENTS表的sid字段。

-- 数据类型: cid为char(10),sid为char(10),money为decimal(10,2)。

create TABLE CARD(
 cid char(10) primary key,
 sid char(10) references students(sid),
 money decimal(10,2)
);

输出

Result 33

on ch.cid = co.cid
join students as st
on st.sid = ch.sid
group by co.cid,st.sid,co.cname
having count(st.sid) >=2

[2025-12-25 10:36:39] 在 590 ms (execution: 256 ms, fetching: 334 ms) 内检索到从 1 开始的 50 行

[2025-12-25 10:37:23] School_Data> create view view_sc as
 select distinct s.sname,co.cname,ch.score
 from students as s
 left join choices as ch
 on s.sid = ch.sid
 left join courses as co
 on co.cid = ch.cid

[2025-12-25 10:37:23] 在 21 ms 内完成

[2025-12-25 10:37:34] School_Data> select * from view_sc

[2025-12-25 10:37:35] 在 1 s 274 ms (execution: 322 ms, fetching: 952 ms) 内检索到从 1 开始的 299,369 行

[2025-12-25 10:38:40] School_Data> create TABLE CARD(
 cid char(10) primary key,
 sid char(10) references students(sid),
 money decimal(10,2)
)

[2025-12-25 10:38:40] 在 24 ms 内完成

代码执行截图

(2) 第2题：给表CARD增加属性列bank

SQL语句：

```
ALTER TABLE card ADD bank char(20);
```

结果截图：



代码执行截图

(3) 第3题：删除表CARD的属性bank

SQL语句：

```
ALTER TABLE card DROP COLUMN bank;
```

结果截图：



代码执行截图

(4) 第4题：查询工资最高的教师姓名和开设的课程编号

SQL语句：

```

SELECT tname,cid
FROM teachers AS t
JOIN choices AS ch
ON ch.tid = t.tid
WHERE salary=(SELECT MAX(salary) FROM TEACHERS WHERE salary IS NOT NULL)

```

结果截图：

The screenshot shows a SQL IDE interface. The top pane displays the following SQL code:

```

--4. 查询工资最高的教师姓名和开设的课程编号。
select tname,cid
from teachers as t
join choices as ch
on ch.tid = t.tid
where salary=(select MAX(salary) from TEACHERS where salary is not null)

```

The bottom pane shows the execution results in a table format with columns 'tname' and 'cid'.

	tname	cid
1	xarwndjtd	10032
2	xarwndjtd	10017
3	wbvmggkvi	10041
4	xarwndjtd	10001
5	xarwndjtd	10026
6	qzftezkyu	10005
7	wbvmggkvi	10047
8	eghgugq	10022
9	qzftezkyu	10012
10	eghgugq	10003
11	qzftezkyu	10048
12	xarwndjtd	10042

The interface also shows a toolbar with various icons and a status bar indicating 84 rows.

代码执行截图

(5) 第5题：查询所有课程平均课时

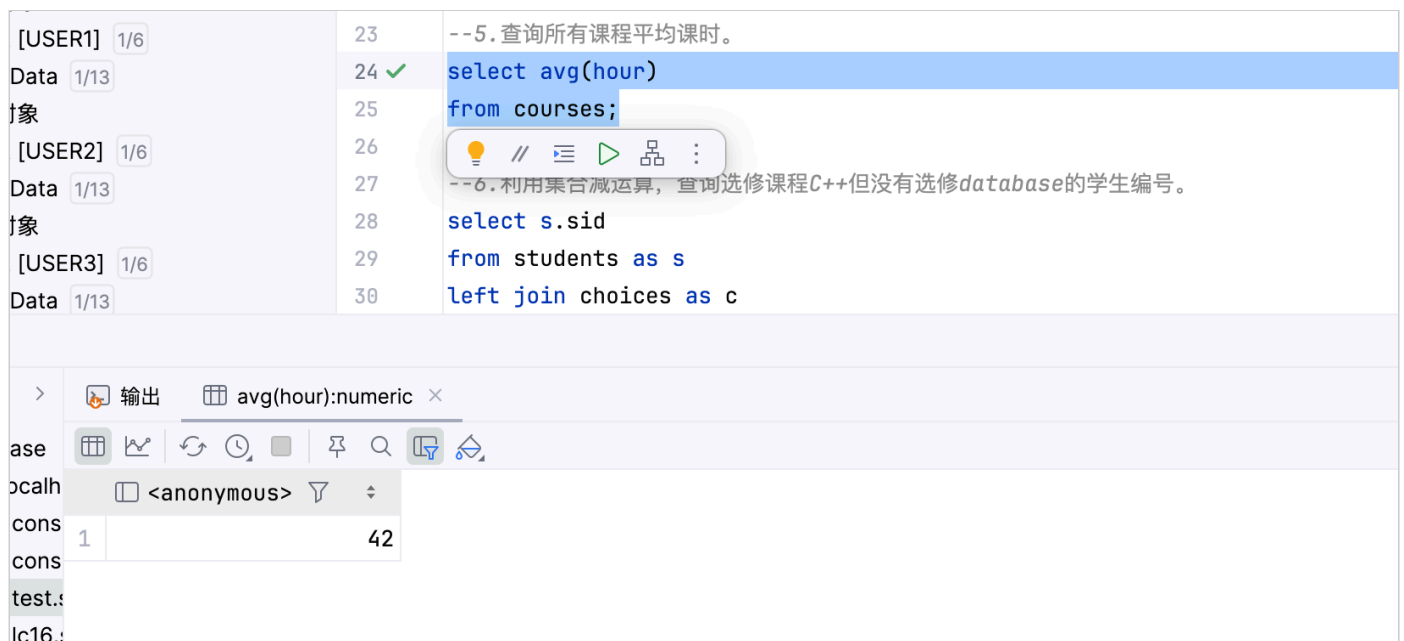
SQL语句：

```

SELECT AVG(hour)
FROM courses;

```

结果截图：



代码执行截图

(6) 第6题：查询选修课程C++但没有选修database的学生编号

SQL语句：

```
SELECT s.sid
FROM students AS s
LEFT JOIN choices AS c
ON s.sid = c.sid
WHERE c.cid = (SELECT cid FROM courses WHERE cname = "sql-string">'C++')

EXCEPT

SELECT s.sid
FROM students AS s
LEFT JOIN choices AS c
ON s.sid = c.sid
WHERE c.cid = (SELECT cid FROM courses WHERE cname = "sql-string">'database')
```

结果截图：

3

3

data 1/3

象

[USER1] 1/6

data 1/13

象

[USER2] 1/6

data 1/13

象

[USER3] 1/6

data 1/13

象

27

28 ✓

29

30

31

32

33

34

35

36

37

38

39

40

41

--6.利用集合减运算，查询选修课程C++但没有选修database的学生编号。

select s.sid

from students as s

left join choices as c

on s.sid = c.sid

where c.cid = (select cid from courses where cname = 'C++')

except

select s.sid

from students as s

left join choices as c

on s.sid = c.sid

where c.cid = (select cid from courses where cname = 'database')

输出 6.利用集合减运算，查询选修课程C++但没有选修database的学生编号。 ×

se

calh

ons

ons

est..

c16..

sid

1 813339734

2 890213390

3 829652792

4 816243464

5 855577639

6 866207466

7 843386504

8 822916698

9 844740581

10 864573288

11 888834544

12 858430359

13 806711144

5,596 行

est.sol

27:1 (3)

代码执行截图

(7) 第7题：查询课时与UML或C++的课时一样的课程名称

SQL语句：

```
SELECT cname FROM COURSES AS c
WHERE c.hour IN (SELECT hour FROM COURSES WHERE cname IN ("sql-string">'UML', "sql-string">'C++') AND hour IS N
```

结果截图：

R1] 1/6

1/13

R2] 1/6

1/13

R3] 1/6

1/13

41

42

43 ✓

44

45

46

47

48

49

50

51

-- 7. 查询课时与UML或C++的课时一样的课程名称。

select cname from COURSES as c

where c.hour in (select hour from COURSES where cname in ('UML', 'C++') and hour is not null)

-- 8. 查询database课程的平均成绩和最高成绩。

select avg(ch.score) as avg_score, max(ch.score) as max_score

from courses as co

join choices as ch

on co.cid = ch.cid

where co.cname = 'database'

输出 7.查询课时与UML或C++的课时一样的课程名称。 x

cnome

1 c++

2 uml

3 data structure

4 computer network

5 asp

6 struts

7 c#

7行

代码执行截图

(8) 第8题： 查询database课程的平均成绩和最高成绩

SQL语句：

```
SELECT AVG(ch.score) AS avg_score, MAX(ch.score) AS max_score
FROM courses AS co
JOIN choices AS ch
ON co.cid = ch.cid
WHERE co.cname = "sql-string">'database'
```

结果截图：

R2] 1/6 40
1/13 47
48 ✓
R3] 1/6
1/13 49
50
51
52
53

```
--8. 查询database课程的平均成绩和最高成绩。  
select avg(ch.score) as avg_score, max(ch.score) as max_score  
from courses as co  
join choices as ch  
on co.cid = ch.cid  
where co.cname = 'database'
```

输出 8. 查询database课程的平均成绩和最高成绩。 ×

avg_score max_score

1	75	99
---	----	----

1行

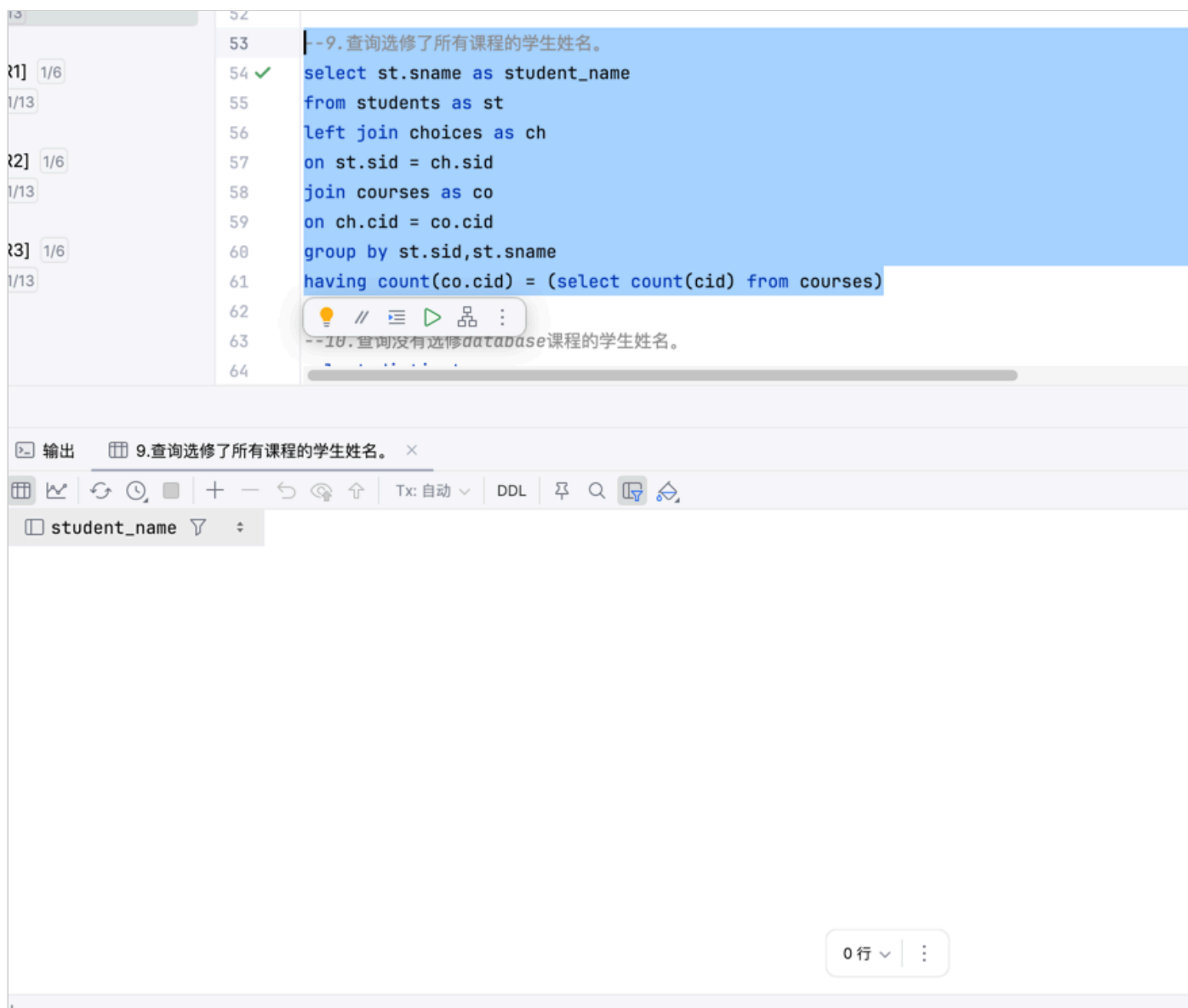
代码执行截图

(9) 第9题：查询选修了所有课程的学生姓名

SQL语句：

```
SELECT st.sname AS student_name  
FROM students AS st  
LEFT JOIN choices AS ch  
ON st.sid = ch.sid  
JOIN courses AS co  
ON ch.cid = co.cid  
GROUP BY st.sid,st.sname  
HAVING COUNT(co.cid) = (SELECT COUNT(cid) FROM courses)
```

结果截图：



代码执行截图

(10) 第10题：查询没有选修database课程的学生姓名

SQL语句：

```
SELECT DISTINCT sname
FROM students

EXCEPT

SELECT DISTINCT s.sname
FROM students AS s
LEFT JOIN choices AS ch
ON s.sid = ch.sid
JOIN courses AS co
ON ch.cid = co.cid
WHERE co.cname = "sql-string">'database'
```

结果截图：

13

1/6

1/13

1/6

1/13

1/6

1/13

64

65 ✓

66

67

68

69

70

71

72

73

74

75

76

77

10. 查询没有选修database课程的学生姓名。

```
select distinct sname
from students

except

select distinct s.sname
from students as s
left join choices as ch
on s.sid = ch.sid
join courses as co
on ch.cid = co.cid
where co.cname = 'database'
```

输出

10. 查询没有选修database课程的学生姓名。

sname

1 ddujgj

2 nsnczjqdl

3 jlgipc

4 lgwfsa

5 vqmhbdzt

6 rosskrmf

7 yqplu

8 vqwdacnrx

9 dqvrax

10 ajmxytoi

11 iykjox

12 teznx

13

94,222 行

代码执行截图

(11) 第11题：查询至少被两位学生选修的课程名称

SQL语句：

```
SELECT DISTINCT co.cname
FROM courses AS co
JOIN choices AS ch
ON ch.cid = co.cid
JOIN students AS st
ON st.sid = ch.sid
GROUP BY co.cid,co.cname
HAVING COUNT(st.sid) >=2
```

结果截图：

3] 1/6
/13

2] 1/6
/13

3] 1/6
/13

77

--11. 查询至少被两位学生选修的课程名称。

78 ✓

select distinct co.cname

79

from courses as co

80

join choices as ch

81

on ch.cid = co.cid

82

join students as st

83

on st.sid = ch.sid

84

group by co.cid,co.cname

85

having count(st.sid) >=2

86

87

--12. 建立视图VIEW_SC, 这个视图由学生姓名以及其选修的课程名称和相应分数构成

88

create view view_sc as

89

select distinct s.sname,co.cname,ch.score

90

from students as s

输出

11.查询至少被两位学生选修的课程名称。

×

cname

1 algorithm

2 architectonics

3 artificial intel...

4 asp

5 basic

6 c

7 c#

8 c++

9 c++/stl program...

10 cobol

11 compiling princ...

12 computer graphi...

13 computer interf...

50 行

⌵

⋮

代码执行截图

(12) 第12题：建立视图VIEW_SC

SQL语句：

```
CREATE VIEW view_sc AS
SELECT DISTINCT s.sname,co.cname,ch.score
FROM students AS s
LEFT JOIN choices AS ch
ON s.sid = ch.sid
LEFT JOIN courses AS co
ON co.cid = ch.cid
```

结果截图：

1/13

R2] 1/6

1/13

R3] 1/6

1/13

88 ✓ 25 ms

89

90 ✓

91

92

93

94

95

96

建立视图VIEW_SC，这个视图由学生姓名以及其选修的课程名称和相应分数构成。

create view view_sc as

select distinct s.sname,co.cname,ch.score

from students as s

left join choices as ch

on s.sid = ch.sid

left join courses as co

on co.cid = ch.cid

输出

11.查询至少被两位学生选修的课程名称。

[2025-12-25 10:48:37] School_Data> --12.建立视图VIEW_SC，这个视图由学生姓名以及其选修的课程名称和相应分数构成。

create view view_sc as

select distinct s.sname,co.cname,ch.score

from students as s

left join choices as ch

on s.sid = ch.sid

left join courses as co

on co.cid = ch.cid

[2025-12-25 10:48:37] 在 8 ms 内完成

代码执行截图

(13) 第13题：利用视图VIEW_SC查询分数大于90的学生姓名

SQL语句：

```
SELECT sname FROM view_sc
WHERE score>90 AND score IS NOT NULL
```

结果截图：

ER2] 1/6
1/13

101
102 ✓
103
104
105
106
107
108

--13.利用视图VIEW_SC，查询分数大于90分数的学生姓名。

select sname from view_sc
where score>90 and score is not null

--14.是否能利用视图VIEW_SC插入数据？为什么？（问答题）

--15.创建规则R1，确保插入的money值大于0，并将规则R1绑定到表CARD的money属性上。

输出 13.利用视图VIEW_SC，查询分数大于90分数的学生姓名。 ×

sname

1	aaalbgivr
2	aakrtd
3	aaowfqrqm
4	aapzouh
5	aatvtsj
6	abcbthfpx
7	abgxvz
8	abhugtb
9	abnxuthks
10	abquq
11	abtuif
12	abuvvbcd
13	abvvdcd

53,946行

代码执行截图

(14) 第14题：是否能利用视图VIEW_SC插入数据？为什么？

答案：

不能，因为：
视图VIEW_SC是基于三个表(students, choices, courses)的left join连接创建的，数据库无法确定如何将插入的数据分配到这三个表中同时因为视图是一种虚拟表，它通过将查询的结果存储在数据库中来提供数据，但不能用于存储数据。
视图的目的是提供一种方便的方式来查询数据库中的数据，而不是用于存储数据。
因此，使用视图时，可以使用 SELECT 语句来查询数据，但不能使用 INSERT 、 UPDATE 或 DELETE 语句来插入、更新或删除数据。

(15) 第15题：创建规则R1并绑定到表CARD的money属性

SQL语句：

```
CREATE RULE R1 AS @value>0  
go  
EXEC SP_BINDRULE R1,"sql-string">'CARD.[MONEY]'
```

结果截图：

```
108 --15. 创建规则R1, 确保插入的money值大于0, 并将规则R1绑定到表CARD的money属性上。
109 create rule R1 as @value>0
110 go
111 exec sp_bindrule R1,'CARD.[MONEY]'
112
113 --16. 在表CARD中插入一条违反规则R1的记录。
114
115
116 --17. 解绑表CARD上规则R1的绑定。

> 输出 13.利用视图VIEW_SC, 查询分数大于90分数的学生姓名。
[2025-12-25 10:48:51] School_Data> select sname from view_sc
                                where score>90
[2025-12-25 10:48:51] 在 483 ms (execution: 90 ms, fetching: 393 ms) 内检索到从 1 开始的 53,946 行
[2025-12-25 10:49:12] School_Data> select sname from view_sc
                                where score>90 and score is not null
[2025-12-25 10:49:12] 在 464 ms (execution: 77 ms, fetching: 387 ms) 内检索到从 1 开始的 53,946 行
[2025-12-25 10:50:04] School_Data> --15. 创建规则R1, 确保插入的money值大于0, 并将规则R1绑定到表CARD的money属性上。
                                create rule R1 as @value>0
                                go
                                exec sp_bindrule R1,'CARD.[MONEY]'
[2025-12-25 10:50:04] [S0001][156] 行 4: Incorrect syntax near the keyword 'exec'.
[2025-12-25 10:50:17] School_Data> create rule R1 as @value>0
[2025-12-25 10:50:17] 在 10 ms 内完成
[2025-12-25 10:50:17] School_Data> exec sp_bindrule R1,'CARD.[MONEY]'
[2025-12-25 10:50:17] [S0001][15514] Rule bound to table column.
[2025-12-25 10:50:17] 在 89 ms 内完成
```

代码执行截图

(16) 第16题：在表CARD中插入违反规则R1的记录

SQL语句：

```
INSERT card VALUES("sql-string">'1',"sql-string">'800001216',1);
INSERT card VALUES("sql-string">'2',"sql-string">'800002933',-100);
```

结果截图：

```
112
113 --16. 在表CARD中插入一条违反规则R1的记录。
114 insert card values('1','800001216',1);
115
116 select sid from students
117 select * from card
118
119 insert card values('2','800002933',-100);
120
121
122
[S0000][513] 行 1: A column insert or update conflicts with a rule imposed by a previous CREATE RULE statement. The statement was terminated. The conflict occurred in
database 'School_Data', table 'dbo.CARD', column 'money'.
```

代码执行截图

(17) 第17题：解绑表CARD上规则R1的绑定

SQL语句：

```
EXEC SP_UNBINDRULE "sql-string">'CARD.[MONEY]'
go
INSERT card VALUES("sql-string">'2',"sql-string">'800002933',-100);
```

结果截图：

1/13

ER2] 1/6

1/13

ER3] 1/6

1/13

120

121

122 ✓

123

124 ✓

125

126

127

128

129

--17.解绑表CARD上规则R1的绑定。

exec sp_unbindrule 'CARD.[MONEY]'

go

insert card values('2','800002933',-100);

select * from card

💡 // ⋮ ▶ 🧩 ⋮

--18.为表STUDENTS建立触发器T1，禁止删除学号为800015960的记录。

🔍 输出

Result 68-2

🧩 📈 ↺ ⌚ ⏹ ⌵ 🔍 🗨 📄

cid sid money

1 1 800001216 1.00

2 2 800002933 -100.00

2行 ⌵ ⋮

代码执行截图

(18) 第18题：为表STUDENTS建立触发器T1

SQL语句：

```
CREATE TRIGGER T1
ON STUDENTS
INSTEAD OF DELETE
AS
BEGIN
  IF EXISTS (SELECT * FROM deleted WHERE sid = "sql-string">'800015960')
  BEGIN
    RAISERROR("sql-string">'Cannot DELETE student with ID 800015960', 16, 1)
    ROLLBACK TRANSACTION
  END
END;
```

结果截图：

3

1] 1/6
/13

2] 1/6
/13

3] 1/6
/13

129 ✓

130

131

132

133

134

135

136

137

138

139

140

141

--18.为表STUDENTS建立触发器T1，禁止删除学号为800015960的记录。

CREATE TRIGGER T1

ON STUDENTS

INSTEAD OF DELETE

AS

BEGIN

IF EXISTS (SELECT * FROM deleted WHERE sid = '800015960')

BEGIN

RAISERROR('Cannot delete student with ID 800015960', 16, 1)

ROLLBACK TRANSACTION

END

END;

输出

11.查询至少被两位学生选修的课程名称。

[2025-12-25 11:08:19] School_Data> drop trigger T1

[2025-12-25 11:08:19] 在 13 ms 内完成

[2025-12-25 11:08:24] School_Data> --18.为表STUDENTS建立触发器T1，禁止删除学号为800015960的记录。

CREATE TRIGGER T1

ON STUDENTS

INSTEAD OF DELETE

AS

BEGIN

IF EXISTS (SELECT * FROM deleted WHERE sid = '800015960')

BEGIN

RAISERROR('Cannot delete student with ID 800015960', 16, 1)

ROLLBACK TRANSACTION

END

END;

[2025-12-25 11:08:24] 在 13 ms 内完成

代码执行截图

(19) 第19题：演示违反触发器T1的操作

SQL语句：

```
DELETE FROM students WHERE sid = "sql-string">'800015960';
```

结果截图：

1/13

1/6

1/13

144	--19. 演示违反触发器T1的操作。
145	<code>delete from students where sid = '800015960';</code>
146	
147	--20. 编写一个嵌套事务。外层修改STUDENTS表某记录，内层在COURSES表插入一条记录。演示

[S0001][50000] 行 9: Cannot delete student with ID 800015960

输出

11.查询至少被两位学生选修的课程名称。

[2025-12-25 11:08:24] School_Data> --18. 为表STUDENTS建立触发器T1，禁止删除学号为800015960的记录。

CREATE TRIGGER T1

ON STUDENTS

INSTEAD OF DELETE

AS

BEGIN

IF EXISTS (SELECT * FROM deleted WHERE sid = '800015960')

BEGIN

RAISERROR('Cannot delete student with ID 800015960', 16, 1)

ROLLBACK TRANSACTION

END

END;

[2025-12-25 11:08:24] 在 13 ms 内完成

[2025-12-25 11:08:46] School_Data> delete from students where sid = '800015960'

[2025-12-25 11:08:46] [S0001][50000] 行 9: Cannot delete student with ID 800015960

代码执行截图

(20) 第20题：嵌套事务演示

说明：执行前将名字先改为MAC

SQL语句：

```
UPDATE students SET sname="sql-string">'MAC' WHERE sid = "sql-string">'800015960';
BEGIN TRANSACTION outer_tran
  BEGIN TRY
    "sql-comment">-- 外层事务：修改students表
    UPDATE students SET sname="sql-string">'MIKE' WHERE sid = "sql-string">'800015960'

    "sql-comment">-- 创建保存点（savepoint）用于部分回滚，替代嵌套事务
    SAVE TRANSACTION inner_savepoint

    BEGIN TRY
      "sql-comment">-- 尝试插入一条可能违反约束的记录（例如：使用已存在的cid）
      "sql-comment">-- 假设"sql-string">'100'已经是一个存在的course ID，这会导致主键冲突
      INSERT INTO courses (cid, cname, hour) VALUES("sql-string">'100',"sql-string">'test',10);
    end TRY
    BEGIN CATCH
      "sql-comment">-- 内层插入失败，回滚到保存点
      ROLLBACK TRANSACTION inner_savepoint
      PRINT "sql-string">'内层插入失败：' + ERROR_MESSAGE()
      "sql-comment">-- 由于内层失败，将外层事务标记为失败
      RAISERROR("sql-string">'内层插入失败，回滚外层事务', 16, 1)
    end CATCH

    COMMIT TRANSACTION outer_tran
```



```
end TRY
BEGIN CATCH
    "sql-comment">-- 外层事务回滚
    ROLLBACK TRANSACTION outer_tran
    PRINT "sql-string">'外层事务回滚: ' + ERROR_MESSAGE()
end CATCH

go

"sql-comment">-- 验证修改是否被回滚
SELECT sname FROM students WHERE sid = "sql-string">'800015960';
```

结果截图：

SQL Server Enterprise Edition (64-bit) - 2017

147 --20. 编写一个嵌套事务。外层修改STUDENTS表某记录，内层在COURSES表插入一条记录。演示内层插入操作失败后，外层修改操作回滚。

148 ✓ begin transaction outer_tran

149 begin try

150 -- 外层事务：修改students表

151 update students set sname='MIKE' where sid = '800015960'

152 end try

153 -- 创建保存点 (savepoint) 用于部分回滚，替代嵌套事务

154 save transaction inner_savepoint

155 end catch

156 raiserror('内层插入失败，回滚外层事务', 16, 1)

157 end catch

158 commit transaction outer_tran

159 end try

160 begin catch

161 -- 外层事务回滚

162 rollback transaction outer_tran

163 print '外层事务回滚: ' + ERROR_MESSAGE()

164 end catch

内层插入失败: Violation of PRIMARY KEY constraint 'PK_COURSES'. Cannot insert duplicate key in object 'dbo.COURSES'. The duplicate key value is (100).

外层事务回滚: 内层插入失败，回滚外层事务

[2025-12-25 11:11:34] 16 ms 中有 1 行受到影响

[2025-12-25 11:11:34] School_Data> select sname from students where sid = '800015960'

[2025-12-25 11:11:34] 在 319 ms (execution: 5 ms, fetching: 314 ms) 内检索到从 1 开始的 1 行

代码执行截图

结果截图：

ER1] 1/6
1/13

ER2] 1/6
1/13

ER3] 1/6
1/13

147

148 ✓

149

150

151

152

153

154

155

156

157

--20. 编写一个嵌套事务。外层修改STUDENTS表某记录， 内层在COURSES表插入一条记录。演示内层插入操作失败后，外层修改操作回滚。

begin transaction outer_tran

begin try

-- 外层事务：修改students表

update students set sname='MIKE' where sid = '800015960'

-- 创建保存点 (savepoint) 用于部分回滚，替代嵌套事务

save transaction inner_savepoint

begin try

-- 尝试插入一条可能违反约束的记录（例如：使用已存在的cid)

输出 验证修改是否被回滚

grid chart refresh undo redo stop + - rotate up

Tx: 自动 DDL 保存 搜索 打印 分享

CSV

sname

1 MAC

1行

结果截图